

Text DSP System

Amir AlMasri¹ 1240890¹

Birzeit University

1240890@student.birzeit.edu

Abstract

This project implements a text DSP system in which each English letter is represented by four simultaneous audio tones. MATLAB and Simulink generate a reusable character signal bank, while Python assembles messages and recovers text with two independent detection methods: Fourier analysis and a bank of frequency-selective filters. Each character occupies 320 samples at 8000 Hz. Both decoders recovered all 771 characters in eight clean WAV test messages. These results establish correct operation for aligned synthetic signals; they do not establish robustness to noise, timing offsets, or acoustic transmission.

Index Terms: multi-frequency signaling, Simulink, FFT, resonator bank.

1. Introduction

Multi-frequency signaling represents symbols by combinations of tones rather than by a spoken voice. The receiver identifies the tones and converts them back to symbols. This project applies that idea to uppercase letters, lowercase letters, and spaces, following the frequency table. It provides a practical comparison between frequency-domain detection and time-domain filtering.

The objective is an end-to-end system that accepts text, generates and plays a WAV file, and independently decodes WAV input. Decoders must use the waveform itself rather than retain the original text. Separate input loading, frequency detection, and character lookup functions make the two methods easy to compare.

2. Problem Specification

The supported alphabet contains 53 symbols: 26 lowercase letters, 26 uppercase letters, and space. Three frequencies identify the letter; a fourth is 100 Hz for lowercase and 200 Hz for uppercase. Space uses the triplet 1000, 2000, and 4000 Hz, and its case component is ignored. The encoder consistently uses 100 Hz for that component.

The encoder and the reported evaluation use a sampling frequency of 8000 Hz and a character duration of 0.04 s. Therefore, each segment contains exactly 320 samples. Characters are concatenated without inserting silence. The receiver assumes that the first sample is a character boundary and that no samples are missing.

Table 1 lists the candidate frequencies. The group names refer to columns of the supplied mapping; they do not imply that every middle frequency is numerically larger than every low frequency.

Table 1. Candidate frequencies

Group	Frequencies in Hz
Case	100, 200
Low	400, 600, 1000
Middle	800, 1200, 2000
High	1600, 2400, 4000

3. Data

The MATLAB script reads a colon-separated frequency file with 27 rows: a to z followed by space. It reuses the Simulink model to generate every character. A MAT file stores a (320, 53) matrix, labels, sampling frequency, and character duration. Columns are ordered a to z, A to Z, then space.

Python loads these columns into a character-to-samples dictionary. A message is assembled by looking up each character and concatenating its samples in order. The waveform is stored as a 32-bit floating-point WAV. For example, the four-character message “Hi a” produces 1280 samples and lasts 0.16 s.

Evaluation uses eight clean messages saved as WAV files: lowercase alphabet (26 characters), uppercase alphabet (26), “Hi a Hello World” (16), “aaAA zzZZ Hello” (18), “ a A ” (7), a single space (1), a repeating letter pattern (101), and a personal message beginning “My name is Amir AlMasri” (576). The total is 771 characters per decoder. Quotation marks are not input characters. The repeated-letter test has two spaces after aaAA and three after zzZZ; the boundary test has two leading spaces, one internal space, and two trailing spaces. The exported Evaluation.csv records each filename, complete expected and recovered text, counts, accuracy, and decoding time.

4. Evaluation Criteria

For each test, the encoder concatenates the stored character samples into a float32 waveform at 8000 Hz, with 320 samples (40 ms) per character and no inserted gaps. The waveform is saved as WAV and read back from that file before decoding. Both methods process the resulting character segments independently. Recovered text is compared with the known input at each position. Letter case and all spaces count; exact recovery requires identical text and length. Results are exported to CSV after decoding.

$$\text{Accuracy (\%)} = 100 \times C / L \quad (1)$$

Here, C is the number of correct characters and L is the input length. Additional checks verify sample rate, sample count, WAV round-trip integrity, and decoding of PCM and floating-point files. Execution time excludes playback and is reported as a diagnostic, not as a hardware-independent speed claim.

5. Approach

5.1. Signal generation and encoding

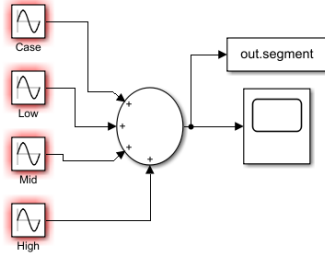


Fig. 1. Simulink character generator.

`Script.m` assigns `fCase`, `fLow`, `fMid`, and `fHigh` for each character and calls `sim` with `ReturnWorkspaceOutputs` enabled. It reads `SimOut.segment`, verifies that the result contains N samples, and places the vector in one column of signals. After all 53 simulations, it saves signals, characters, F_s , and T_c in `character_signals.mat`. This file is the interface between the Simulink generator and the Python application.

Four Simulink Sine Wave blocks use phase $\pi/2$ so their outputs are cosines. Each has amplitude 0.25, zero bias, and sample time $1/8000$ s. Their outputs are summed. The amplitude choice bounds the absolute sum by one and avoids clipping when writing conventional normalized audio. A phase of $\pi/2$ converts each sine output to a cosine.

$$x[n] = 0.25 \cos(2\pi f_1 n / F_s) + 0.25 \cos(2\pi f_2 n / F_s) + 0.25 \cos(2\pi f_3 n / F_s) + 0.25 \cos(2\pi f_4 n / F_s) \quad (2)$$

Here, the four frequencies correspond to case and the three letter groups, and n runs from 0 to 319. The simulation stop time is $319/8000$ s because sample zero is included. The 320 samples represent a 40 ms segment even though the final sample timestamp is 39.875 ms.

Cosine generation is essential at the Nyquist endpoint: a 4000 Hz cosine sampled at 8000 Hz alternates between positive and negative values, whereas a zero-phase sine at that frequency vanishes at the sampling instants. Figure 2 shows a representative uppercase H, whose four tones are 200, 400, 2000, and 2400 Hz.

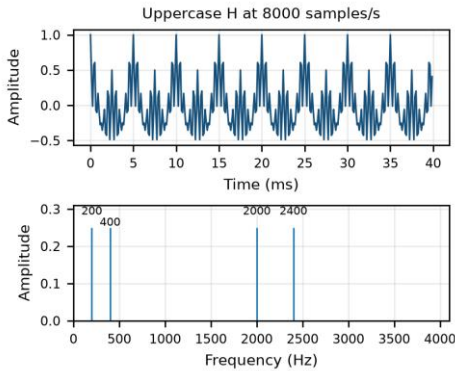


Fig. 2. Time samples and one-sided amplitude spectrum of uppercase H.

The Python encoder rejects empty input and unsupported characters. It concatenates the validated character arrays, saves the resulting WAV, and optionally plays it. Playback uses the original sample rate; reducing that rate changes both pitch and duration and is not used for decoding.

5.2. Fourier decoder

Both decoders use the sampling frequency F_s read from the WAV file rather than requiring every input file to be sampled at 8000 Hz. Each character segment contains $N = \text{round}(F_s \times T_c)$ samples. This supports other sampling rates that can represent the required tones, provided $T_c = 0.04$ s, character boundaries are correctly aligned, and the tones follow the frequency lookup table. Resampling is not required solely because the input sampling rate differs from 8000 Hz. At 8000 Hz, the WAV is divided into rows of 320 samples. `NumPy rfft` computes the discrete Fourier transform for real input, returning 161 bins from DC to 4000 Hz. The bin spacing is $8000/320 = 25$ Hz. Every specified tone falls exactly on a bin, so its index is the frequency divided by 25. For example, 100 and 200 Hz correspond to indices 4 and 8. More generally, the bin spacing is $F_s/N = 1/T_c = 25$ Hz when $N = F_s \times T_c$ is an integer, so these indices remain valid at other sampling rates with the same character duration.

The detector selects the largest magnitude separately within each group in Table 1. It does not select the four largest bins globally, which could omit a group. Raw FFT magnitudes suffice for these clean decisions; the plotted one-sided amplitude spectrum uses normalization and does not double the DC or Nyquist bins.

Each segment produces a row containing case, low, middle, and high frequencies. The shared message extractor searches the frequency table for the matching triplet and applies uppercase when the detected case tone is 200 Hz. Space is unaffected by case conversion. The characters are joined without stripping leading or repeated spaces.

5.3. Filter bank decoder

The second method applies ten second-order resonators and one highpass filter to each segment. Resonators are designed with `SciPy iirpeak` at candidate frequencies below 4000 Hz, using the sampling frequency read from the input WAV file. The highpass branch is also designed using that sampling frequency. Each uses a 25 Hz -3 dB bandwidth and quality factor shown in equation (3). Thus the 100 Hz resonator has $Q = 4$ and the 2400 Hz resonator has $Q = 96$.

$$Q = f_0 / BW \quad (3)$$

The numerator coefficients b and denominator coefficients a are passed to `scipy.signal.lfilter` to apply the corresponding difference equation. Each character is processed from zero filter state, preventing the preceding character from influencing the next decision. The initial response is nevertheless part of the measured energy.

The 4000 Hz endpoint is handled by a second-order Butterworth highpass with a 3900 Hz cutoff, represented as second-order sections and applied with `sosfilt`. This avoids using an ordinary bandpass resonator at Nyquist. The endpoint design is a practical exception to the otherwise resonant bank.

For each branch, the detector sums the squared output samples. It chooses the branch with maximum energy within each group, produces the same four-frequency row as the FFT detector, and reuses the message extractor. No FFT is used in the filter-bank decision.

$$E = y[0]^2 + y[1]^2 + \dots + y[319]^2 \quad (4)$$

6. Results and Analysis

6.1. Clean message accuracy

Table 2 summarizes the eight WAV files tested by both decoders, giving 16 runs. Both methods recovered every message exactly: 771/771 correct characters (100%) and 8/8 complete messages per decoder. The long personal message lasts 23.04 s; its recorded decoding times are 15.96 ms for FFT and 54.04 ms for the filter bank. These are single-run timings, not a benchmark.

Table 2. Correct characters over total characters

Test message	FFT	Filters
Lowercase alphabet	26/26	26/26
Uppercase alphabet	26/26	26/26
Mixed case	16/16	16/16
Repeated letters and spaces	18/18	18/18
Boundary spaces	7/7	7/7
Single space	1/1	1/1
Repeating pattern	101/101	101/101
Personal message	576/576	576/576
Total	771/771	771/771
Accuracy	100%	100%

The FFT decision benefits from exact alignment of the prescribed frequencies with the DFT grid. The filter-bank decision also separates these clean tones at the selected bandwidth. Figure 3 shows the designed responses, including the separate branch at the upper endpoint. The interface can display each resonator's output waveform, spectrum, and pole-zero diagram.

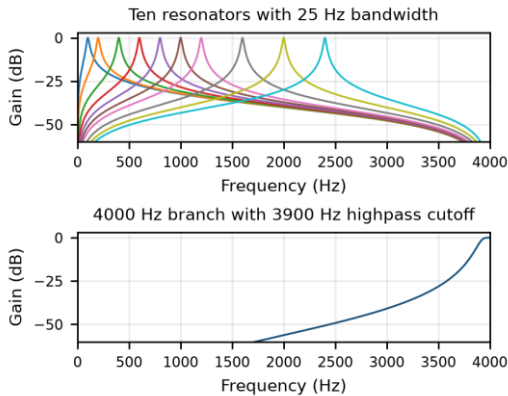


Fig. 3. Frequency responses of the ten resonators and the 4000 Hz highpass branch.

6.2. Ringing and residual components

A resonator output need not appear as a pure sinusoid during a 40 ms segment. Switching from zero state to a multi-tone input excites a startup transient. Finite stopband attenuation also allows small residual components from other tones. The resulting irregular trace is deterministic and should not automatically be described as random noise.

Independent vertical autoscaling can make a weak nonmatching branch look as large as a matching branch. Figure 4 therefore uses identical amplitude limits. For uppercase H, the 2000 Hz branch is a matching detector, while the 1200 Hz branch is not. Comparing energy rather than visual trace height avoids this plotting ambiguity.

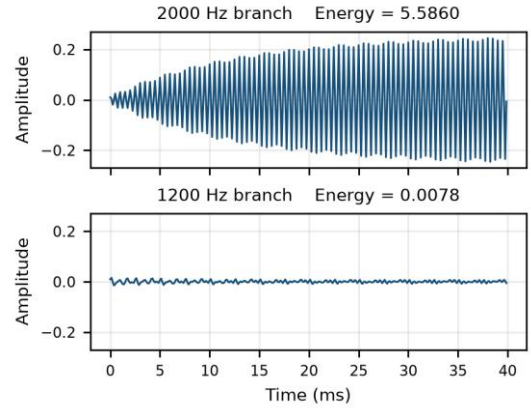


Fig. 4. Matching and nonmatching resonator outputs for uppercase H on a common amplitude scale.

6.3. Limits of the evaluation

The 25 Hz bandwidth worked for the measured clean messages, but no bandwidth sweep established an optimum. Narrower filters improve frequency selectivity while increasing settling time; wider filters settle faster but admit more adjacent-frequency energy. An appropriate future comparison would vary bandwidth while holding messages and evaluation criteria fixed.

The tests share the same character generator and perfect 320-sample boundaries. They do not measure performance after timing shifts, dropped samples, frequency drift, resampling, background noise, clipping, or an acoustic channel. The 4000 Hz tone is particularly sensitive to the endpoint characteristics of playback and recording hardware. Reliable digital WAV decoding does not imply reliable speaker-to-microphone transmission.

Maximum-energy selection always chooses a candidate. Therefore, decoder agreement alone is not proof of correctness for an unlabeled WAV. Invalid or weak signals may need confidence thresholds and synchronization logic; these features are not part of the demonstrated detector.

7. Development

Development began with one lowercase character in Simulink and a sample-count check. The same model was then reused to produce the complete signal bank. Separating the character data from the encoder allowed Python to generate new messages without rerunning Simulink for every input.

A critical filter implementation issue was corrected before evaluation: `irpeak` returns numerator `b` followed by denominator `a`. The earlier reversed assignment caused `lfilter` to receive the coefficients in the wrong roles. The measured results in this report use the corrected assignment. There is no recorded before-and-after accuracy comparison, so the correction is not presented as a quantified accuracy improvement.

The interface provides text or WAV input, a decoder selector, playback and WAV saving, and selectable plots. An evaluation control executes the labelled test suite and exports results. The shared lookup and message-extraction code keep character interpretation consistent between the two methods, while their frequency detectors remain independent.

For visual checking, decoded text is re-encoded using the stored character bank. Figure 5 compares the original ‘Hi a’ signal with reconstructions from the FFT and filter-bank decoder text in both the time and frequency domains. The waveforms and spectra show close agreement.” Keep the verified numerical error statement afterward.

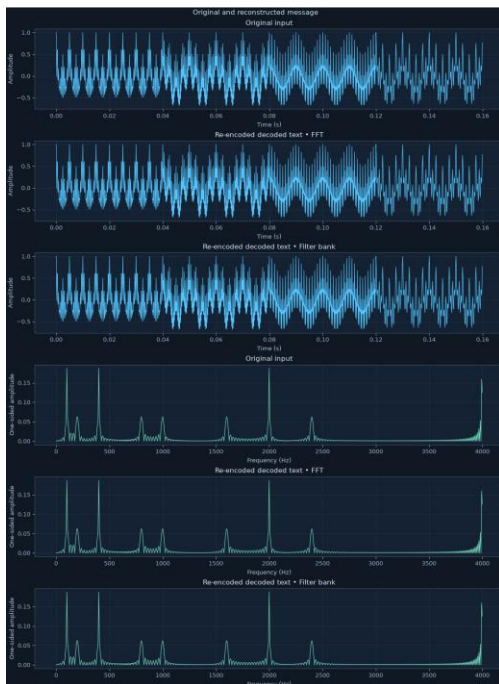


Fig. 5. Original ‘Hi a’ signal and reconstructions from both decoders in the time and frequency domains.

8. Conclusions

The completed system demonstrates text-to-audio encoding and audio-to-text recovery using two distinct DSP methods. A reusable Simulink signal bank supports Python message generation, and both the FFT and filter-bank decoders recovered every character in the 771-character clean evaluation set. The implementation also preserves letter case and repeated spaces. The experiment highlights the distinction between time samples and frequency bins, the need to compare candidates within their assigned groups, and the transient behavior of narrow resonators. Explicit treatment of the 4000 Hz Nyquist component is required in both generation and filtering. Further work should evaluate bandwidth choices, noise and timing sensitivity, and instructor-provided files before making broader reliability claims.

9. Appendix

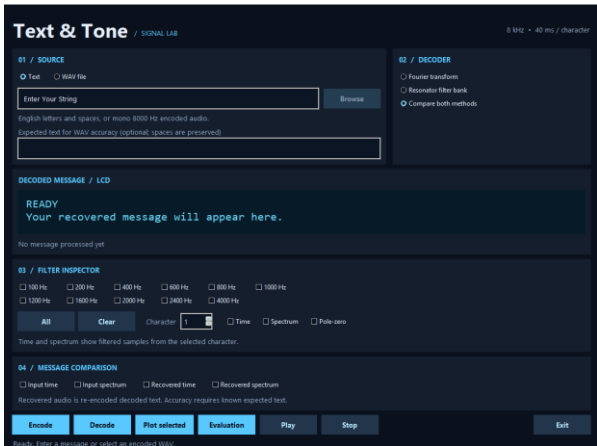


Fig. 6. Project user interface.

Decoder evaluation

Clean synthetic messages from your character bank includes case and spaces.
A baseline test, not a measurement of noise tolerance or instructor file accuracy.

FFT: 100.00% (614/614) | Filter bank: 100.00% (614/614)

Test	Decoder	Accuracy	Correct / total	Time (ms)	Result
Lowercase alphabet	FFT	100.00%	26/26	1.07	PASS
Lowercase alphabet	Filter bank	100.00%	26/26	3.44	PASS
Uppercase alphabet	FFT	100.00%	26/26	0.94	PASS
Uppercase alphabet	Filter bank	100.00%	26/26	3.19	PASS
Mixed case	FFT	100.00%	16/16	0.83	PASS
Mixed case	Filter bank	100.00%	16/16	3.81	PASS
Repeated letters and spaces	FFT	100.00%	18/18	0.73	PASS
Repeated letters and spaces	Filter bank	100.00%	18/18	2.30	PASS
Boundary spaces	FFT	100.00%	7/7	0.27	PASS
Boundary spaces	Filter bank	100.00%	7/7	1.06	PASS
Space	FFT	100.00%	1/1	0.07	PASS
Space	Filter bank	100.00%	1/1	0.35	PASS
Long message	FFT	100.00%	520/520	13.95	PASS
Long message	Filter bank	100.00%	520/520	70.05	PASS

Export CSV

Fig. 7. Evaluation table.

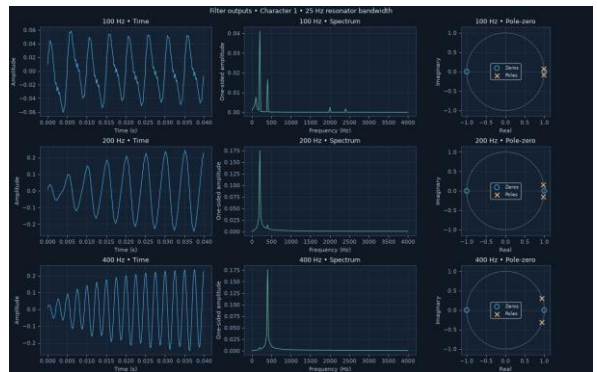


Fig. 8. Example filter output and pole-zero diagram.